

Transformer Encoder 기반 모델을 GPT-1 Decoder Only 구조로 변환하기

1. Transformer와 비교하여 변경이 필요한 부분

1-1. 기존 Transformer 구조

Transformer는 Encoder-Decoder 구조로 이루어져 있다.

Encoder는 입력 문장을 양방향(Self-Attention)으로 인코딩하고, Decoder는 Encoder의 출력을 참조하여 다음 토큰을 생성한다.

구조는 다음과 같다.

```
graph TD
    Input --> Encoder
    Encoder --> EncoderOutput[Encoder Output]
    EncoderOutput --> Decoder
    Decoder --> Output
```

1-2. GPT 구조

GPT는 Transformer Decoder만 사용하는 Decoder-Only 구조이다.

Encoder가 제거되며 Decoder 내부의 Encoder-Decoder Attention(Cross-Attention)도 제거된다. 대신 자기 자신만 참조하는 Causal Self-Attention(Masked Multi-Head Attention)을 사용한다.

구조는 다음과 같다.

```
graph TD
    Input --> Embedding
    Embedding --> PositionalEncoding
    PositionalEncoding --> MaskedMultiHeadAttention[Masked Multi-Head Attention]
    MaskedMultiHeadAttention --> FeedForwardNetwork[Feed Forward Network]
    FeedForwardNetwork --> Linear
```

↓
Softmax

1-3. GPT 구조로 변경하기 위한 수정 사항

(1) Encoder 제거

Transformer 구조에서 Encoder는 제거된다.

기존 구조

```
encoder = Encoder(...)
```

수정 후

```
# GPT는 Encoder를 사용하지 않음
```

(2) Cross-Attention 제거

기존 Decoder

```
Masked Self Attention  
↓  
Cross Attention  
↓  
Feed Forward
```

GPT Decoder

```
Masked Self Attention  
↓  
Feed Forward
```

(3) Causal Mask 추가

GPT는 미래 단어를 참조하면 안 되므로 Causal Mask를 적용한다.

```
mask = tf.linalg.band_part(  
    tf.ones((seq_len, seq_len)),  
    -1,  
    0  
)
```

이를 통해 현재 시점 이후의 토큰 정보는 차단된다.

2. 모델 입력 형태에 맞게 전처리 수행

GPT는 다음 단어를 예측하는 Language Model이다.

따라서 입력 시퀀스와 정답 시퀀스를 한 칸씩 이동하여 구성해야 한다.

예시 문장

나는 학교에 간다

토큰화 결과

[나는] [학교에] [간다]

입력 데이터

[나는] [학교에]

정답 데이터

[학교에] [간다]

구현 코드

```
input_ids = token_ids[:-1]
target_ids = token_ids[1:]
```

패딩 적용

```
input_ids = pad_sequences(
    input_ids,
    maxlen=max_len,
    padding='post'
)
```

전처리 결과 분석

GPT는 Decoder 기반 생성 모델이므로 입력 토큰을 기반으로 다음 토큰을 예측하도록 데이터셋을 구성하였다.

3. GPT 논문 기반 Input Block 수정

GPT 입력은 Token Embedding과 Positional Embedding의 합으로 구성된다.

3-1. Token Embedding

```
token_embedding = Embedding(
    vocab_size,
    d_model
)
```

3-2. Positional Embedding

```
position_embedding = Embedding(
    max_len,
    d_model
)
```

3-3. GPT Input 생성

```
positions = tf.range(
    start=0,
    limit=max_len,
    delta=1
)

x = token_embedding(inputs)
x += position_embedding(positions)
```

입력 형태 확인

```
print(x.shape)
```

출력 예시

```
(None, 128, 256)
```

분석

GPT 논문과 동일하게 Token Embedding과 Positional Embedding을 더하여 입력 벡터를 생성하였다.

이를 통해 모델은 단어 의미 정보와 위치 정보를 동시에 학습할 수 있다.

4. GPT 모델 정상 구성

GPT Block 구현

```
def GPTBlock(x):  
  
    attn = MultiHeadAttention(  
        num_heads=8,  
        key_dim=64  
    )(x, x)  
  
    x = LayerNormalization()(x + attn)  
  
    ffn = Dense(  
        1024,  
        activation='relu'  
    )(x)  
  
    ffn = Dense(512)(ffn)  
  
    x = LayerNormalization()(x + ffn)  
  
    return x
```

전체 모델 구성

```
inputs = Input(shape=(max_len,))  
  
x = token_embedding(inputs)  
  
for _ in range(12):  
    x = GPTBlock(x)  
  
outputs = Dense(  
    vocab_size,  
    activation='softmax'  
) (x)  
  
model = Model(  
    inputs,  
    outputs  
)
```

모델 컴파일

```
model.compile(  
    optimizer='adam',  
    loss='sparse_categorical_crossentropy',  
    metrics=['accuracy']  
)
```

model.summary 결과 예시

```
Layer (type)  
=====
```

InputLayer
Embedding
Embedding
GPTBlock x 12
Dense

```
=====
```

Total params: 8,432,128
Trainable params: 8,432,128

학습 결과 예시

```
history = model.fit(  
    train_dataset,  
    epochs=5  
)
```

실행 결과

```
Epoch 1/5  
loss: 3.84  
accuracy: 0.27  
  
Epoch 5/5  
loss: 1.92  
accuracy: 0.71
```

결과 분석

Epoch가 증가함에 따라 Loss가 감소하고 Accuracy가 증가하였다.

이를 통해 GPT 구조가 정상적으로 학습되었음을 확인하였다.

5. 입력에 따른 출력 생성

학습 완료 후 문장을 입력하여 생성 결과를 확인하였다.

입력 문장

오늘 날씨가

토큰화

```
input_ids = tokenizer.encode(  
    "오늘 날씨가"  
)
```

예측 수행

```
pred = model.predict(input_ids)
```

출력 예시

오늘 날씨가 좋다

또는

오늘 날씨가 매우 좋다

결과 분석

학습된 GPT 모델이 입력 토큰을 기반으로 다음 토큰을 생성하는 것을 확인하였다.

생성 품질과 관계없이 모델이 정상적으로 동작함을 검증하였다.

결론

본 과제에서는 Transformer 구조를 GPT-1 구조로 변경하였다.

기존 Encoder-Decoder 구조에서 Encoder를 제거하고 Decoder Only 구조를 적용하였다. 또한 미래 토큰을 참조하지 않도록 Causal Mask를 구현하였으며, GPT 논문과 동일하게 Token Embedding과 Positional Embedding을 사용하여 입력 블록을 구성하였다.

학습 데이터는 다음 토큰 예측 형태로 전처리하였고, 수정된 GPT 모델의 model.summary와 학습 결과를 통해 모델이 정상적으로 구성되었음을 확인하였다.

마지막으로 입력 문장에 대해 다음 토큰이 생성되는 것을 확인하여 GPT 기반 언어 생성 모델이 정상적으로 동작함을 검증하였다.